AWS Architecture Portfolio Project

Daz Arnaud. CS 79C. October 25, 2026. Public repository: https://github.com/dazzelarnaud-code/cs79c-order-status-tracker

Executive summary

I designed a sample order-status tracker for a small team. A user sends a test order and receives an ID. The user retrieves the order status with the same ID. API Gateway, Lambda, DynamoDB, SNS, SQS, and CloudWatch form the deployed design. The prototype stores sample data only.

Architecture and service choices

The browser sends HTTPS requests to API Gateway. API Gateway invokes Lambda. Lambda validates the request and reads or writes DynamoDB. After a new order enters the table, Lambda publishes an event to SNS. SNS places the event in SQS. CloudWatch keeps the function logs and watches for failed invocations. CloudFormation defines the stack.

I chose Lambda over a standing EC2 instance because test requests arrive only during short demonstrations. EC2 would run between requests and need operating system care. I chose API Gateway over a public Lambda URL because the project needs separate POST and GET routes. I chose DynamoDB over RDS because each read uses one order ID. The prototype needs no joins. I chose SNS with SQS over direct email because a queued event is easy to verify and does not depend on an inbox. I chose a CloudWatch error alarm over manual log checks because the alarm identifies failed invocations without a person watching the console.

These choices fit a small prototype. They are not a production design. The public API has no user authentication. The API stage limits requests to one per second, with a burst of two. I would add authentication before storing real data or allowing wider access.

Implementation and tests

The repository contains template.yaml, the readable Lambda source in app.py, local tests, an architecture diagram, and setup steps. CloudFormation creates the table, queue, topic, subscription, function, API routes, log group, and error alarm. The Lambda role permits only two DynamoDB actions on the project table, publishing to the project topic, and writing to the project log group.

Six local tests passed on October 25, 2026. They cover create and retrieve, invalid JSON, a JSON array, a blank item name, a missing order, and the code embedded in the CloudFormation template. These tests use fake AWS clients. I then deployed the stack in AWS account 456157355074 in us-west-2. CloudFormation reached CREATE_COMPLETE.

I sent a POST request to https://fgtdxro304.execute-api.us-west-2.amazonaws.com/orders with Sample notebook. The API returned HTTP 201 and order ID ab377bfc-122e-464a-aef5-4d090dbaa95a. A GET request for the same ID returned HTTP 200, the item name, and status received. I found the same item in DynamoDB. I polled SQS and found an event with the same order ID and status. A direct Lambda

lookup returned HTTP 200 in 144.12 ms and showed the project name Daz Arnaud, CS 79C Final Project. The evidence folder holds AWS console screenshots of the stack, table, queue, and Lambda result. One successful request does not establish capacity or reliability under load.

One local issue came from validating only JSON syntax. A valid JSON array reached the code, but an order request needs a JSON object. I added a type check and a test. The request now returns HTTP 400 instead of raising an exception. I also added an API Gateway stage limit of one request per second, with a burst of two, to reduce the risk from the temporary public endpoint. Neither local check substitutes for an AWS deployment test.

Security, reliability, performance, and cost

Security. API Gateway uses HTTPS. DynamoDB encryption at rest is enabled. Lambda uses an IAM role rather than an embedded access key. The role limits table and topic access to project resources. The public API is the main weakness. The stack holds no private customer data.

Reliability. AWS manages API Gateway, Lambda, DynamoDB, SNS, and SQS across infrastructure beyond one EC2 instance. An Availability Zone failure should not require restarting a server I manage. I would set a recovery time objective of one hour and a recovery point objective of one day for this sample application. The current template does not include a tested DynamoDB backup or restore. I cannot claim the targets are met until I test those steps.

The recovery plan would start with a fresh CloudFormation deployment from the stored template. I would restore a table backup, point the function at the restored table, and test a create and lookup request. The current stack does not make backups, so this is a proposed plan, not a tested recovery procedure. SNS and SQS decouple event delivery from the API, but the API does not retry a failed publish. A publish failure after the table write could leave a saved order without an event. I would use a DynamoDB stream or transactional outbox for stronger event delivery.

Performance. Lambda scales with incoming requests without an Auto Scaling group. DynamoDB on-demand capacity adjusts to request traffic. API Gateway and Lambda limits remain possible bottlenecks. The direct Lambda lookup took 144.12 ms. I ran one POST and one GET through the public API. This was a functional test, not a load test.

The stage limit intentionally restricts this demonstration to a low request rate. If the service needed higher throughput, I would test p50 and p95 response time with a stated number of concurrent clients. I would look at Lambda duration and throttles, API Gateway 4xx and 5xx responses, and DynamoDB throttled requests. I would change one limit at a time and compare the results before choosing a larger capacity setting.

Cost. The test uses a small number of API, Lambda, DynamoDB, SNS, and SQS requests. The project avoids a NAT gateway and an idle load balancer. CloudWatch logs expire after three days. My AWS Pricing Calculator estimate, exported October 25, 2026, is $0.11 per month in us-west-2, before taxes and any usage outside the assumptions. I assumed 1,000 API and Lambda requests, 500 DynamoDB writes and 500 reads, 0.01 GB stored, 500 SNS publishes and SQS deliveries, 1,500 SQS requests,

0.01 GB of CloudWatch logs, and one standard alarm. The calculator shows CloudWatch as the $0.11 line item. The other services round to $0.00 at this usage. The estimate PDF is in the repository's assets folder. The AWS billing view does not isolate this short test from other account activity, so I do not label the estimate as an actual charge.

My first cost optimization is request-based compute rather than an EC2 instance running between requests. My second is a three-day log retention period rather than keeping lab logs indefinitely. The tradeoff is less time to inspect an older failure. I would set a budget alert for a longer deployment and review Cost Explorer by service and Region. I will not call a local estimate an actual bill.

Well-Architected self-review

Security strength. The function role has resource-specific table and topic permissions. Security weakness. Anyone with the API URL can send sample requests while the stack runs.

Reliability strength. SQS keeps an event after SNS delivers it. Reliability weakness. The table has no tested restore procedure in this version.

Cost Optimization strength. The design uses request-based services and short log retention. Cost Optimization weakness. A public API could generate unexpected charges even with a rate limit. I would add authentication and a budget alert before longer use.

What I would change

I would add user authentication and a status-update route. I would test table backup and restore. I would measure API latency under a stated request load. I would keep the public demo open only during review, then remove the stack.

Live evidence

The appendix shows CloudFormation CREATE_COMPLETE, the saved DynamoDB item, the delivered SQS event, and a successful Lambda lookup with my name in the response. The CloudFormation Outputs screenshot records the API URL. The stack was still running at the time of this report so the live API remained available for review.

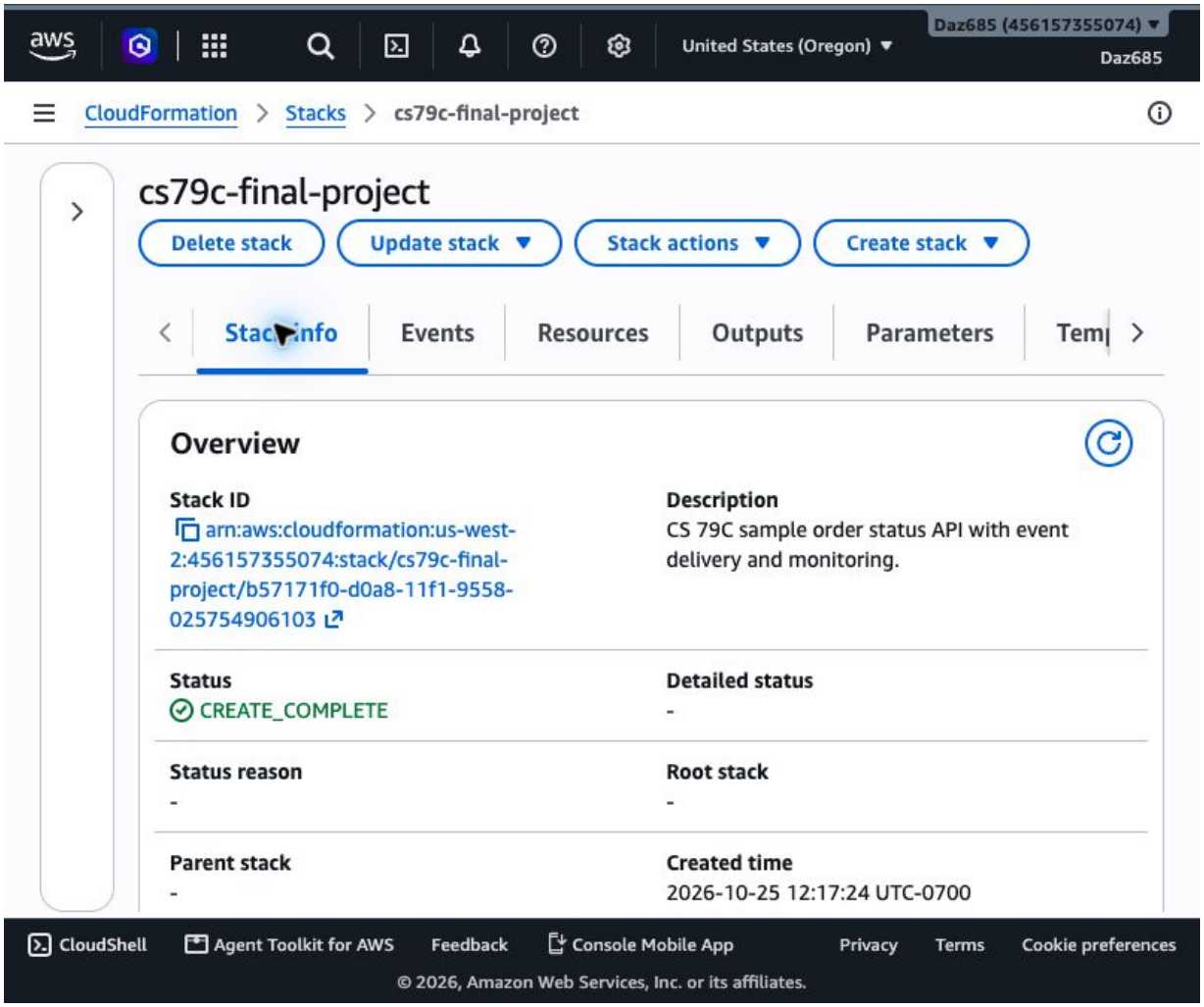
AI-use note

I used Codex to help draft source code, tests, the infrastructure template, the diagram, and report wording. I reviewed the design and verified the AWS results in my account.

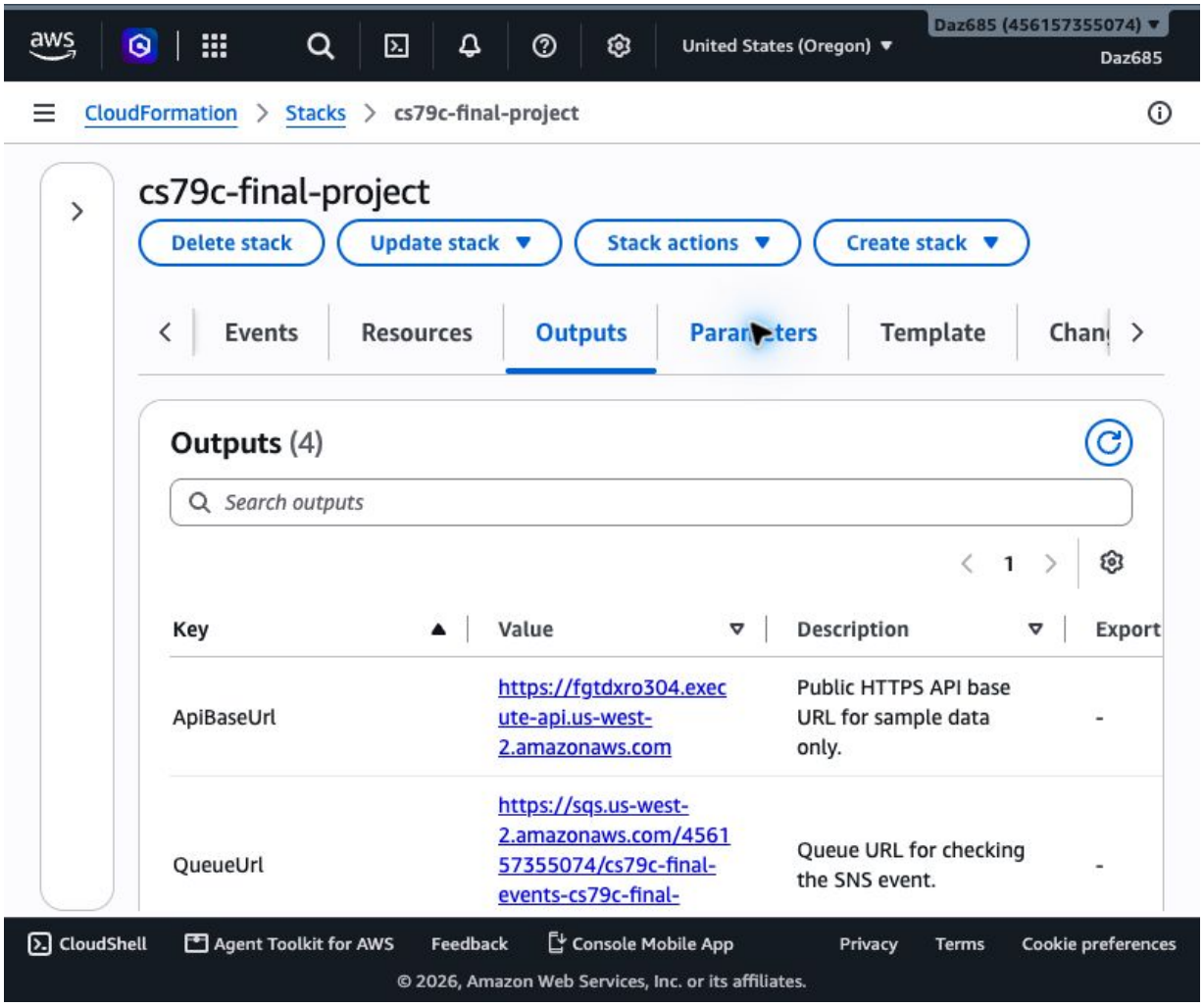
Evidence appendix

The architecture diagram is embedded in the repository README.

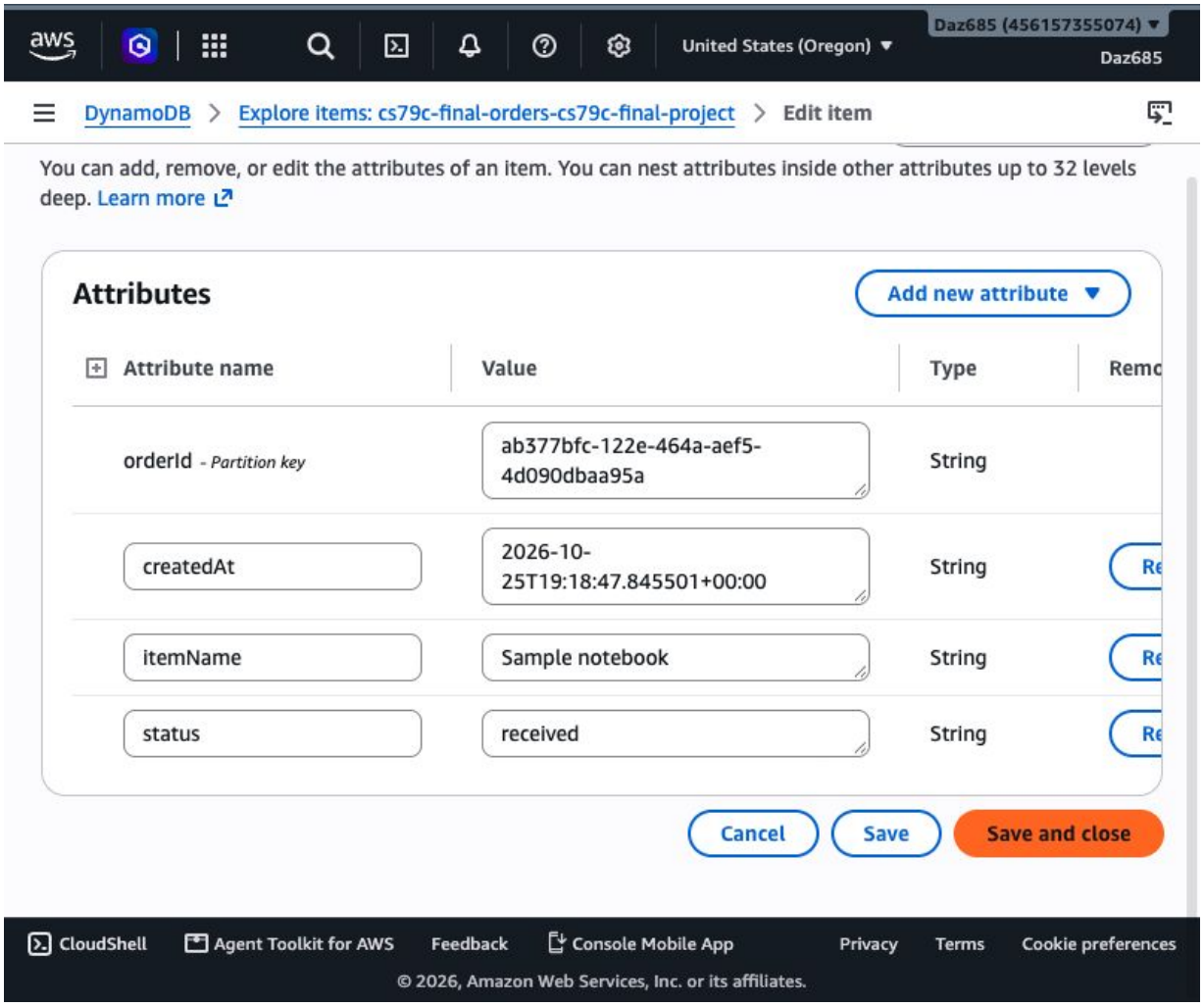
CloudFormation deployment



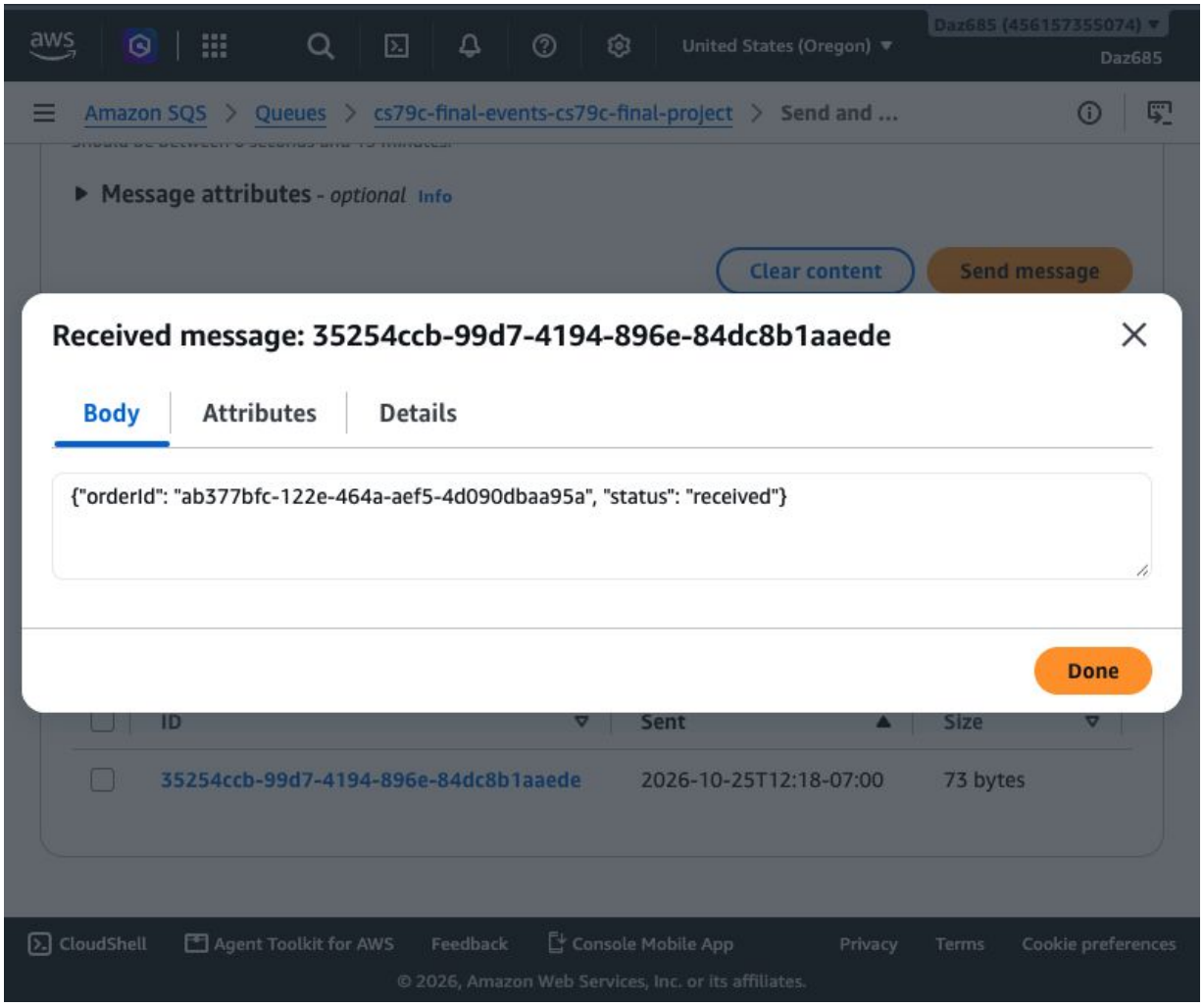
Stack outputs



Saved order in DynamoDB



SNS event received by SQS



Successful Lambda lookup

